

Introduction and Problem Space

Software verification is incredibly useful because it mathematically guarantees that a program works correctly. However, it isn't used everywhere because it requires a massive amount of manual human effort. Writing code in a verification language like Dafny actually feels pretty normal, much like standard programming, but figuring out the proofs is a huge, time-consuming chore. Even for a simple program (for example, a 15-line stack tree traversal), a developer has to carefully figure out and write down complex invariants, requires clauses, and post-conditions just so the automated solver can prove the code is right with these invariants being much longer than the actual code. Trying to remove this heavy burden on developers has driven decades of research, pushing the field from purely manual math toward modern, AI-assisted tools.

Advancements in Verification

The theoretical foundation for program correctness was established by Hoare in 1969. By utilizing preconditions, programs, and results, Hoare demonstrated how to prove correctness mathematically, though this required human insight to formulate intermediate assertions and there was no way to automatically check these proofs for correctness. Decades later, tools like Dafny (2010) automated the proof-checking process utilizing SMT solvers. Despite this automation, developers were still forced to manually write the required invariants, leaving a massive burden on the programmer when scaling to complex, real-world systems.

To address this, researchers began integrating machine learning. Early black-box ML models struggled because verifiers provided only binary pass/fail feedback, leaving the models stuck guessing. This changed with the ICE framework (2014), which proved that ML models could synthesize invariants by learning from a verifier's examples, counterexamples, and implications.

The introduction of Generative Language Modeling (2020) marked a turning point. Researchers discovered that Transformer models custom-trained on logic languages like Metamath could successfully predict formal reasoning steps. However, these early generative models hallucinated heavily without a strict formal checker to guide them. This limitation led to the development of Lemur (2024), a hybrid approach that placed LLMs into a fully automated loop. In this system, LLMs rapidly proposed invariants, while automated SMT solvers rigorously validated them and filtered out hallucinations. Lemur successfully outperformed traditional verifiers on complex benchmarks, though it struggled with non-deterministic behavior and output formatting issues.

Most recently, the field has shifted toward human-in-the-loop assistance with Lean Copilot (2025). Rather than running automated loops in the background, Lean Copilot embeds a local language model directly into the proof assistant. It suggests mathematically sound tactics to the developer in real-time, instantly filtering AI hallucinations through Lean's kernel before the human even sees them.

Open Research Questions

While AI models now provide powerful search heuristics alongside formal verifiers, several open research questions remain. The current major challenge is teaching flexible language models to natively speak the strict languages of mathematical verifiers. Future work must focus on translating dense SMT solver errors into actionable LLM prompts, overcoming LLM context window limits during iterative proof refinement, and scaling verification from simple arrays to highly complex, heap-allocated data structures.

Annotated References

1. Hoare, C. A. R. (1969). An Axiomatic Basis for Computer Programming.

In his 1969 paper, Hoare confronted the problem that computer science lacked a rigorous mathematical foundation to formally guarantee that a program would execute correctly as intended. To solve this, Hoare introduced an axiomatic basis utilizing preconditions, programs, and results (now known as "Hoare tuples") to mathematically verify code. The paper evaluates this solution theoretically, providing mathematical proofs that demonstrate how intermediate assertions can be logically combined to prove the correctness of foundational algorithms.

However, a major limitation of this approach is that it requires a large amount of human insight and manual mathematical effort to create those intermediate assertions. Future work, therefore, needed to focus on developing automated solvers to reduce this manual cognitive burden.

2. Leino, K. R. M. (2010). Dafny: An Automatic Program Verifier for Functional Correctness.

Addressing the tedious nature of manual proofs, K.R.M. Leino confronted the need for automated checking tools to verify functional correctness. The solution was the introduction of Dafny, which is a verification-aware programming language that automates proof checking by leveraging an SMT solver. The paper evaluates Dafny's effectiveness through case studies, demonstrating how the language can successfully verify the functional correctness of programs ranging from a simple compute sum function to a complex stack-based tree traversal. Despite this automation, a key limitation is that Dafny still acts as a "human bottleneck," because developers are forced to manually supply all necessary invariants, requires clauses, and post-conditions for the SMT solvers to actually work, so it takes someone a long time to learn

and become proficient in Dafny. Future work is needed to focus on generating these invariants automatically to easily scale verification to complex, real-world systems.

3. Garg, P., et al. (2014). ICE: A Robust Framework for Learning Invariants.

Early machine learning models struggled to generate programmatic invariants because traditional verifiers only provided binary pass/fail feedback, leaving the black-box models stuck blindly guessing without actionable data. To solve this problem, Garg and colleagues developed the ICE framework, a robust learning paradigm that synthesizes invariants by learning from a richer feedback loop consisting of examples, counterexamples, and implications. The researchers evaluated their solution by implementing the ICE-learning algorithms within a verification tool, providing mathematical proofs demonstrating that the iterative learning process is strongly convergent on benchmark invariant problems. While highly successful at the time, a limitation is that this pre-LLM approach relied heavily on basic decision tree and SMT solver learning rather than the more flexible generative reasoning seen in modern models. Future work shifted toward generative architectures capable of broader logic prediction.

4. Polu, S., & Sutskever, I. (2020). Generative Language Modeling for Automated Theorem Proving.

Theorem proving traditionally suffered from an over-reliance on manual human input or rigid, brute-force solver searches. Polu and Sutskever solved this by applying generative language modeling to automated theorem proving, custom-training a transformer model directly on the formal logic steps of the Metamath language. The researchers evaluated the model's performance on a Metamath testing suite, measuring proof length and accuracy, and discovered that the model

occasionally generated proofs that were actually shorter and easier to understand than those written by human experts. However, a significant limitation was that the model suffered from heavy hallucinations when operating without a strict formal checker and frequently made mistakes on complex proofs requiring many small steps. Future work is required to combine these generative capabilities with strict automated SMT filters.

5. Wu, H., et al. (2024). Lemur: Integrating Large Language Models in Automated Program Verification.

Addressing the persistent hallucination issues of raw LLMs, Wu and co-authors confronted the challenge of deploying language models on complex algorithmic verification tasks. Their solution was Lemur, a hybrid framework that creates a fully automated loop where the LLM rapidly proposes or repairs invariants, while all steps are checked with an SMT solver. The team evaluated Lemur through rigorous performance measurements on complex datasets, showing it solved 107 problems on the Code2Inv benchmark and 25 problems on SV-COMP, significantly outperforming traditional verifiers. Despite these benchmark victories, limitations include non-deterministic behavior that causes identical proofs to randomly succeed or fail, and struggles with output formatting, such as inserting proof markers on the wrong lines of code. Future work involves shifting away from fully automated background loops to improve the interactive developer experience.

6. Song, P., Yang, K., & Anandkumar, A. (2025). Lean Copilot: Large Language Models as Copilots for Theorem Proving in Lean.

Recognizing that fully automated background loops offer a frustrating and opaque developer experience, Song confronted the need for real-time proof assistance. This paper solves this by shifting to an interactive, human-in-the-loop paradigm with Lean Copilot, which embeds a local language model directly into the Lean proof assistant to suggest mathematically sound tactics. The solution was evaluated using both performance metrics and human-centric measurements, demonstrating that Lean Copilot reduced the average number of human-entered tactics (from 3.86 to 2.08) and boosted autonomous theorem-proving rates from 24.4% (with traditional AESOP) to 63.7%. While highly effective at filtering hallucinations before the user sees them, limitations remain regarding context window constraints and handling un-tokenizable math syntax. Future work aims to translate dense SMT errors into actionable LLM prompts for even better interactive refinement.